Robustness of Turkish Punctuation Restoration under Informal Writing Noise

Project Proposal

Kagan Can
CMPE 586 Course Project

Abstract

Punctuation restoration models may rely on spelling and capitalization cues that are unreliable in informal writing. We will study Turkish case loss, Turkish character folding, and internal transpositions using deterministic perturbations. A published punctuation BERT will provide an external reference; matched clean and augmented BERTurk training will isolate the augmentation recipe. A case-loss removal arm will examine that component without redistributing its dose. Development on validation data will precede a frozen held-out evaluation of clean accuracy, noise robustness, and punctuation-class behavior. The contribution will be an auditable experimental study, without assuming an improvement or proposing a new architecture.

Document status (23 October 2026). Written after project completion as a retrospective reconstruction of its research design. Prospective wording is for the proposal format, not an original pre-experiment submission or preregistration. Performance results are excluded; detailed evaluation choices evolved during the project.

1 Motivation and research questions

Punctuation restoration assigns punctuation marks to text in which those marks are absent. A model trained on consistently written text may encounter a different surface form when users omit capital letters, replace Turkish characters with Latin alternatives, or mistype letters. For a cased model, these changes can remove useful boundary cues and alter subword segmentation. Turkish also requires locale-sensitive casing: capital `I` maps to dotless `ı`, while capital `İ` maps to `i`. A generic lowercasing rule would conflate distinct operations.

We will ask three linked questions:

1. **RQ1: Sensitivity.** How does punctuation performance change with noise family and severity, relative to each model's own clean performance?
2. **RQ2: Augmentation.** Does matched noise-augmented fine-tuning improve performance on perturbed inputs, and what clean-text trade-off accompanies it?
3. **RQ3: Case-loss component.** How does removing case loss from training augmentation affect clean, case-loss, and other noisy conditions?

Our hypotheses are that corruption will generally reduce F1, augmentation will reduce at least some of that loss, and retaining case loss will be most helpful under case-loss evaluation. These are empirical expectations, not promised outcomes: negative gains, non-monotonic severity effects, and clean-performance costs will also be reported. We will not assign target F1 values in advance.

Preprint.

2 Related work and intended contribution

Kurt and Çayır [1] provide a Turkish punctuation restoration study, released data, and pretrained checkpoints. Their release will supply the task setting and an external reference model. Our controlled training arms will instead start from the same saved BERTurk initialization [2], so their difference can be interpreted under a shared experimental recipe.

Punctuation restoration and noise-aware training already intersect: Sunkara et al. [3] study punctuation and truecasing for medical ASR and use augmentation to improve robustness to ASR errors. We will adapt the general motivation to writing noise; we will preserve every original word slot rather than importing ASR insertion/deletion alignment rules. Karpukhin et al. [4] motivate synthetic-noise augmentation in machine translation, but their findings do not establish its effect on Turkish punctuation. Náplava et al. [5] study noise modeled from user-generated errors. Our proposed perturbations will be controlled synthetic stress conditions, without claiming that their frequencies represent Turkish users.

The contribution will be a Turkish-focused comparison with deterministic word-level perturbations, matched clean/augmented training, a case-loss removal arm, and transparent evaluation limits. We will not claim architectural novelty, state-of-the-art performance, or the first combination of punctuation restoration and noise augmentation.

3 Data and task formulation

We will use the train, validation, and test CSV splits released with Kurt and Çayır's work [1], recording the exact upstream revision and file hashes. Training will update model parameters; validation will support development and clean-checkpoint selection; the held-out test will be reserved for the final frozen comparisons. We will retain the released split membership and audit malformed records, label validity, lengths, and exact input overlaps. Rows containing empty word slots will be quarantined as whole records with their IDs and exclusion reasons recorded; targets will not be shifted to repair them.

Each original word will receive one of four labels: `O`, `PERIOD`, `COMMA`, or `QUESTION_MARK`, describing punctuation immediately after it. For token classification, supervision will be attached to the first subword only; continuation subwords, special tokens, and padding will use ignored label `-100`. Changed text will be tokenized again while original word IDs, order, counts, and targets remain fixed. Any missing alignment or sequence exceeding the 512-token limit will stop preparation rather than silently truncate examples.

Dataset licensing, exact split provenance, and document independence are unresolved in the available project record. We will document these limits and avoid redistributing raw data. An exact-input-overlap sensitivity analysis will remove test inputs matching released train or validation inputs and rescore the same predictions. This will address one observable overlap issue, not prove independence or rule out prior exposure of the published model.

4 Proposed method

4.1 Controlled perturbations

Table 1 defines the four evaluation families. Single-family mild, medium, and heavy budgets will select exactly $\lfloor pN \rfloor$ eligible positions, where N is the number of original word positions and $p \in \{0.02, 0.05, 0.10\}$. Stable hash ranking will select positions without labels or model predictions; insufficient eligibility will raise an error. Seeds 586, 587, and 588 will give paired corruption realizations, with nested position masks across severities.

Full-case-loss and full-folding endpoints will be reported separately from the budgeted grid. We will record realized component edits and final union coverage. Equal component budgets will not be described as equal information loss, and combined noise will not be interpreted as an equal-dose interaction experiment.

Table 1: Planned writing-noise operators. Punctuation targets will stay unchanged.

Family	Operation and control
Case loss	Turkish-aware lowercasing of selected words with a case change.
Turkish folding	Explicit case-preserving mapping of ç, ğ, ı, ö, ş, ü to c, g, i, o, s, u and uppercase counterparts.
Internal swap	One adjacent unequal interior-letter swap in an eligible alphabetic word of at least four letters; first and last letters preserved.
Combined	Reuse component plans; apply swap, case loss, then folding. The union can exceed the single-family changed-word budget.

4.2 Models and matched training

A published Turkish punctuation BERT [1] will provide an external reference with an uncontrolled training history. The primary controlled comparison will examine the training recipe difference between two BERTurk arms: **Clean**, using original inputs, and **Augmented**, replacing selected rows with noisy views. Both will use the same saved encoder and freshly initialized four-label head, training seed 586, eligible records, shuffled row order, optimizer settings, and update budget. All parameters will be fine-tuned. The published reference will not be treated as an untrained encoder or an initialization-matched baseline.

The fixed recipe will use three full epochs, AdamW with learning rate 2×10^{-5} and weight decay 0.01 excluding biases and LayerNorm, a physical batch of eight with four-step accumulation, linear 10% warmup and decay, gradient clipping at 1.0, and FP32 execution on the audited Colab GPU runtime. Cross-entropy will be unweighted over all four labels, including `O`, and normalized by supervised word count across each accumulation window. Three epochs will define a feasible matched budget, not a claim of convergence or hyperparameter optimality.

For each augmented epoch, combined-medium word plans will be built on the training split, then exactly half the rows (rounded down) selected by label-independent hash ranking for noisy views. Rows will be replaced in place, without appending examples. Plans and selected rows will refresh each epoch using training-specific seeds. Some selected views may remain unchanged; we will record this rather than force edits or resample. The row fraction will not be reported as a changed-word rate.

Every arm will complete the same budget. Its checkpoint will be the completed epoch with highest *clean-validation punctuation macro-F1*, choosing the earlier epoch on an exact tie. Noisy validation will diagnose robustness rather than choose the checkpoint. If selected epochs differ, we will also report a matched final-epoch sensitivity comparison. Shared exposure and updates will not imply equal wall-clock time or FLOPs because tokenization lengths can differ.

4.3 Case-loss removal ablation

A third BERTurk training arm will omit case loss while retaining the same saved initialization, row masks, ordering, remaining component plans, and budget. The omitted dose will not be reassigned to folding or swaps. We will compare it with the full augmentation arm on clean input and each medium-noise family using corruption seed 586. This asks what changes when that component is removed from this recipe. Since total corruption also decreases, it will not establish an equal-dose ranking of component importance.

5 Evaluation and analysis plan

After validation-based development, we will freeze model identities, selected checkpoints, input hashes, noise plans, cohorts, and metric definitions before project test scoring. The intended main matrix has 39 conditions per main model: one clean condition, four families at three severities and three seeds, and two full-loss endpoints. Three main models therefore require 117 forward conditions. Five ablation conditions will give 122 in total; existing matched reference predictions will be reused. Primary and overlap-filtered cohorts will be scored from these same predictions,

without re-noising or additional forward passes. This detailed plan is reconstructed here, not claimed as an original pre-project commitment.

The primary metric will be

$$F_{\text{macro}} = \frac{1}{3}\left(F_{1,\text{PERIOD}} + F_{1,\text{COMMA}} + F_{1,\text{QUESTION_MARK}}\right).$$

The `O` class will be omitted only from the punctuation average: its positions will still contribute false positives and false negatives. We will report per-class precision, recall, F1, supports, punctuation micro-averages, and four-class confusion matrices. Robustness loss will compare each model with its own clean score. Augmentation gains will be computed on paired inputs for each corruption seed, then summarized by mean and sample standard deviation. These seeds will measure corruption variation for fixed models, not training variation, statistical significance, or equivalence.

We will inspect edited words, unchanged words in edited contexts, and untouched rows separately, alongside deterministic error samples retained locally. These diagnostics will describe where errors occur without attributing them to a proven attention mechanism. We will publish the full planned condition matrix, including unfavorable contrasts, without pooling unrelated conditions into a single headline score. Test results will not trigger checkpoint or recipe selection; later test-informed changes will be exploratory and need genuinely unseen data for strong independent confirmation.

6 Feasibility, risks, and deliverables

The core scope will fit one student project: one encoder family, three matched fine-tunings, one training seed, and a bounded evaluation matrix. A short technical calibration will check memory and throughput before any authorized long run. Runtime and source checks will fail closed on mismatch. If interrupted, runs will resume only from a compatible recorded checkpoint; incompatible changes will receive new identities rather than overwrite runs. Single-seed uncertainty and synthetic-only evaluation will remain explicit limitations. We will also avoid presenting preserved targets as proof that every corrupted string has a unique linguistic reading.

Table 2 gives a relative work plan, not historical completion dates or a new schedule for rerunning the completed experiments.

Table 2: Staged project plan and completion criteria.

Phase	Planned output / completion criterion
1. Preparation	Literature review; pinned inputs; data and alignment audit; versioned noise operators and shared initialization.
2. Training	Runtime calibration; clean and augmented runs; clean-validation checkpoint selection; compatible model archives.
3. Development	Validation robustness grid; matched case-loss removal run and analysis; explicit budget and selection limits.
4. Held-out study	Frozen evaluation protocol; complete condition matrix; overlap sensitivity and class/context diagnostics.
5. Reporting	Reproducible aggregate tables, figures, final report, source repository, and verified backups before scoped storage cleanup.

Code and configuration will be versioned; immutable receipts will bind source, input, model, and output hashes. Notebooks will orchestrate shared Python functions and be committed with execution disabled and outputs cleared. Raw data, model weights, predictions, private course metadata, and row-level examples will remain outside public Git. Original code licensing will be separate from external dataset, model, and template terms. Success will mean a complete, interpretable comparison, even if a hypothesis is unsupported.

Preparation disclosure. AI assistance was used to draft and format this retrospective proposal and check its consistency with project records. This document neither reports new experiments nor changes the completed project's recorded protocols.

References

[1] Uygar Kurt and Aykut Çayır. 2023. Transformer Based Punctuation Restoration for Turkish. *8th International Conference on Computer Science and Engineering (UBMK)*, 169–174. https://doi.org/10.1109/UBMK59864.2023.10286690. Release: https://github.com/uygarkurt/Turkish-Punctuation-Restoration.

[2] Stefan Schweter. 2020. BERTurk: BERT models for Turkish. https://github.com/stefan-it/turkish-bert.

[3] Monica Sunkara, Srikanth Ronanki, Kalpit Dixit, Sravan Bodapati, and Katrin Kirchhoff. 2020. Robust Prediction of Punctuation and Truecasing for Medical ASR. *NLP for Medical Conversations*, 53–62. https://aclanthology.org/2020.nlpmc-1.8/.

[4] Vladimir Karpukhin, Omer Levy, Jacob Eisenstein, and Marjan Ghazvininejad. 2019. Training on Synthetic Noise Improves Robustness to Natural Noise in Machine Translation. *W-NUT*, 42–47. https://aclanthology.org/D19-5506/.

[5] Jakub Náplava, Martin Popel, Milan Straka, and Jana Straková. 2021. Understanding Model Robustness to User-generated Noisy Texts. *W-NUT*, 340–350. https://aclanthology.org/2021.wnut-1.38/.